

Системне Програмування

З використанням мови програмування **Rust.**
Fundamentals. Standard Enums

Складні типи

Інколи, (дуже часто) не можливо виразити логіку програми за допомогою стандартних типів, таких як:

i8, i16, i32, i64, i128, u8, u16, u32, u64, u128, f32, f64

unit

[]

String, &str,

Tuple,

Named Tuple,

Struct

Приклади такого коду...

```
fn min(xs: &[i32]) -> i32
```

Приклади такого коду...

```
fn index_of(x: i32, xs: &[i32]) -> usize
```


Приклади такого коду...

```
fn read_file(name: String) -> String
```

Приклади такого коду...

```
fn whatever(color: &str) { todo!() }
```

```
fn test() {  
    whatever(color: "red");  
    whatever(color: "Red");  
    whatever(color: "RED");  
    whatever(color: "yellow");  
    whatever(color: "YELLOW");  
    whatever(color: "Green");  
    whatever(color: "Green");  
}
```

скільки існує варіантів помилитися і як буде виглядати наш код...

Або...

$$ax^2 + bx + c = 0$$

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
fn solve_quadratic1(a: f32, b: f32, c: f32) -> ???
```

скільки існує варіантів? Якій тип має повернути функція?

Enum (Enumeration)

В Rust, enum — це тип, який дозволяє визначити набір варіантів (значень), кожен з яких може мати різну структуру та типи даних. Він широко використовується для опису даних, що можуть бути різних типів або станів, і дозволяє спростити роботу з ними за допомогою паттерн-матчингу.

Приклади використання Enum

```
enum Color {  
    Red,  
    Yellow,  
    Green,  
}  
  
fn whatever1(color: Color) {}  
  
fn test() {  
    whatever1(Color::Green);  
    whatever1(Color::Blue);  
}
```

Приклади використання Enum

```
enum Switch {  
    On,  
    Off,  
}  
  
enum Option1 {  
    Enabled,  
    Disabled,  
}
```


Приклади використання Enum

```
enum Message {  
    Quit,  
    Move { x: i32, y: i32 },  
    Write(String),  
    ChangeColor(i32, i32, i32),  
}
```

Приклади такого коду...

```
fn min(xs: &i32) -> i32
```

```
enum Option<A> {  
    None,  
    Some(A),  
}
```

```
fn min(xs: &i32) -> Option<i32>
```


Приклади такого коду...

```
fn index_of(x: i32, xs: &[i32]) -> usize
```

```
enum FoundResult<A> {  
    NotFound,  
    Found(A),  
}
```

```
fn index_of(x: i32, xs: &[i32]) -> FoundResult<usize>
```

Приклади такого коду...

```
fn read_file(name: String) -> String
```

```
enum IOResult<E, A> {  
    Error(E),  
    Result(A)  
}
```

```
fn read_file(name: String) -> IOResult<String, String>
```


Приклади такого коду...

```
fn whatever(color: &str) { todo!() }

fn test() {
  whatever(color: "red");
  whatever(color: "Red");
  whatever(color: "RED");
  whatever(color: "yellow");
  whatever(color: "YELLOW");
  whatever(color: "Green");
  whatever(color: "Green");
}
```

```
enum Color {
  Red,
  Yellow,
  Green,
}

fn whatever(color: Color)

fn test() {
  whatever(Red);
  whatever(Green);
  whatever(Blue);
}
```

Приклади використання Enum

```
enum AuthenticationResult {  
    Token(String),  
    InvalidUsername,  
    InvalidPassword,  
    PasswordExpired,  
    UserIsLocked,  
}  
  
fn auth(username: String, password: String) -> AuthenticationResult
```


Квадратне рівняння

$$ax^2 + bx + c = 0$$

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
fn solve_quadratic1(a: f32, b: f32, c: f32) -> ???
```

- $b^2 - 4ac < 0$ - No roots
- $b^2 - 4ac = 0$ - One Root
- $b^2 - 4ac > 0$ - Two Roots

Квадратне рівняння

- $b^2 - 4ac < 0$ - No roots
- $b^2 - 4ac = 0$ - One Root
- $b^2 - 4ac > 0$ - Two Roots

```
enum Solution {  
    NoRoots,  
    OneRoot(f32),  
    TwoRoots(f32, f32),  
}
```


Квадратне рівняння

```
fn solve_quadratic(a: f32, b: f32, c: f32) -> Solution {  
    use Solution::*;  
  
    let d: f32 = b.powi(n: 2) - 4.0 * a * c;  
  
    if d < 0. {  
        NoRoots  
    } else if d == 0. {  
        OneRoot(-b / (2. * a))  
    } else {  
        let dq: f32 = f32::sqrt(d);  
        let a2: f32 = a * 2.;  
        TwoRoots((-b - dq) / a2, (-b + dq) / a2)  
    }  
}
```

```
enum Solution {  
    NoRoots,  
    OneRoot(f32),  
    TwoRoots(f32, f32),  
}
```

Standard Enums

Option<T> — Definition

- Represents an optional value without using null.
- Forces explicit handling of absence at compile time.

```
pub enum Option<T> {  
    None,  
    Some (T) ,  
}
```

Option<T> — Usage Example

- Used when a value may or may not exist.
- Common in lookups, parsing, and configuration.

```
fn find_user(id: u32) -> Option<String> {  
    if id == 1 { Some("Alice".into()) } else { None }  
}
```


Result<T,E> — Definition

- Encodes success or failure with explicit error type.
- Enables structured error propagation using ?.

```
pub enum Result<T, E> {  
    Ok(T) ,  
    Err(E) ,  
}
```

Result<T,E> — Usage Example

- Used in all fallible operations.
- Prevents silent failures and lost error context.

```
fn divide(a: i32, b: i32) -> Result<i32, &'static str> {  
    if b == 0 { Err("division by zero") }  
    else { Ok(a / b) }  
}
```


Ordering — Definition

- Represents the result of a three-way comparison.
- Fundamental for sorting and Ord implementations.

```
pub enum Ordering {  
    Less, Equal, Greater  
}
```

Ordering — Usage Example

- Used in custom sorting logic.
- Ensures consistent and total ordering semantics.

```
words.sort_by(|a, b| a.len().cmp(&b.len()));
```


Poll<T> — Definition

- Core enum for asynchronous state machines.
- Indicates readiness or pending state.

```
pub enum Poll<T> {  
    Ready(T),  
    Pending,  
}
```

Poll<T> — Usage Example

- Used by Future implementations.
- Prevents blocking in async runtimes.

```
match future.poll() {  
    Poll::Ready(val) => println!("Done"),  
    Poll::Pending => println!("Waiting"),  
}
```


ControlFlow<B,C> — Definition

- Encodes structured early exit from computation.
- Useful in iterators and traversal APIs.

```
pub enum ControlFlow<B, C> {  
    Continue(C),  
    Break(B),  
}
```

ControlFlow — Usage Example

- Allows breaking with a value.
- More expressive than boolean flags.

```
for x in data {  
    if x < 0 { return ControlFlow::Break(x); }  
}
```


std::io::ErrorKind — Definition

- Classifies I/O errors in a portable way.
- Abstracts OS-specific error codes.

`ErrorKind::NotFound`

`ErrorKind::PermissionDenied`

`ErrorKind::TimedOut`

ErrorKind — Usage Example

- Used to branch behavior based on failure type.
- Improves cross-platform reliability.

```
match e.kind() {  
    ErrorKind::NotFound => create_file(),  
    _ => return Err(e),  
}
```


SeekFrom — Definition

- Specifies file cursor movement relative to position.
- Used in low-level file operations.

```
pub enum SeekFrom {  
    Start(u64),  
    End(i64),  
    Current(i64),  
}
```

SeekFrom — Usage Example

- Enables reading from specific file offsets.
- Prevents manual offset miscalculations.

```
file.seek(SeekFrom::End(-10))?;
```


IpAddr — Definition

- Represents IPv4 or IPv6 in a type-safe way.
- Prevents mixing address families incorrectly.

```
pub enum IpAddr {  
    V4 (Ipv4Addr) ,  
    V6 (Ipv6Addr) ,  
}
```

IpAddr — Usage Example

- Allows structured matching on IP versions.
- Avoids storing IPs as raw strings.

```
match ip {  
  IpAddr::V4(v4) => println!("IPv4"),  
  IpAddr::V6(v6) => println!("IPv6"),  
}
```